PRODUCT REQUIREMENTS DOCUMENT (PRD)

Tirumala Engineering College — Integrated College Management Portal

Project Name: Tirumala Engineering College Management Portal
Institution: Tirumala Engineering College, Narasaraopet, Andhra Pradesh
Document Type: Product Requirements Document (PRD)
Version: 1.0
Status: Initial Proposal — For Review and Approval
Prepared By: Project Development Team
Proposed Technology: React.js, Spring Boot, Supabase PostgreSQL, Google Cloud, and Cloudflare Pages

---

1. EXECUTIVE SUMMARY

The objective of this project is to design and develop a modern, secure, responsive, and centralized College Management Portal for Tirumala Engineering College, Narasaraopet.

The platform will provide a unified digital environment for students, faculty members, Heads of Departments (HODs), and the Principal. Each role will have a dedicated dashboard with features and permissions appropriate to its responsibilities.

The portal will initially focus on the following functions:

- Student login and academic information.
- Semester and supplementary examination information.
- Student attendance and attendance percentage tracking.
- Faculty timetable and subject assignments.
- Faculty attendance management through spreadsheet uploads.
- Department-level monitoring by HODs.
- College-wide oversight by the Principal.
- Student and faculty account management.
- Fee information and integration with the college's existing payment system, subject to technical verification.
- Secure backend APIs, database management, and role-based access control.
- Future migration of authorized existing student and academic records.

The proposed system will use React.js for the frontend, Spring Boot for the backend, Supabase PostgreSQL for persistent storage, Google Cloud for backend hosting, and Cloudflare Pages for frontend deployment.

The development strategy is to build the core application first using fictional test data. Once the college's existing data sources and payment platform have been identified and access is authorized, the required records and integrations will be introduced without redesigning the entire application.

The overall objective is to simplify academic administration, reduce repetitive manual work, improve access to information, and provide a reliable digital experience for the college community.

2. PROJECT VISION

To establish a centralized, user-friendly, and secure digital platform that connects students, faculty, HODs, and the Principal through a common academic information system.

The portal should make information available to the appropriate people without exposing unnecessary or unauthorized data.

The platform should be designed to support future improvements, additional academic workflows, and integration with existing college systems.

2.1 Core Principles

1. One centralized platform for the institution.
2. Separate dashboards for different user roles.
3. Simple workflows for everyday academic activities.
4. Spreadsheet-based attendance management to reduce repetitive data entry.
5. Automatic attendance calculations and student-specific updates.
6. Secure authentication and authorization.
7. Compatibility with authorized existing college records.
8. Minimal duplication of existing college processes.
9. Responsive design for desktops, laptops, tablets, and mobile devices.
10. Maintainable architecture that supports future development.

3. PROJECT OBJECTIVES

The project aims to achieve the following objectives:

- Provide students with convenient access to their academic information.
- Allow faculty members to manage their assigned classes, subjects, and attendance.
- Enable faculty to upload attendance spreadsheets instead of manually entering every record through the website.

- Automatically associate attendance records with the correct students using their roll numbers and verified academic assignments.
- Allow HODs to monitor attendance percentages and academic activity across their departments.
- Give the Principal a college-wide overview of departments, HODs, faculty, and relevant student information.
- Provide authorized workflows for adding and managing student records.
- Establish a secure foundation for future student-data migration.
- Investigate and preserve the college's existing fee-payment workflow where technically feasible.
- Conduct functional, integration, and security testing before production deployment.

4. USER ROLES AND ACCESS CONTROL

The platform will have four primary user roles.

4.1 Student

Students will have an individual login and a personal dashboard.

Login

- Student ID: College roll number.
- Initial password: College roll number, if approved by the institution.
- Students must be required to change the initial password on their first login.
- The system must support account recovery through an approved verification process.

Roll numbers will identify students but will not be treated as secret authentication credentials.

Student Dashboard

Students should be able to view:

- Personal student profile.
- Roll number and assigned department.
- Academic year and semester information.
- Enrolled subjects.
- Semester examination information.
- Supplementary examination information.
- Attendance percentage.
- Subject-wise attendance where applicable.
- Monthly attendance summaries.

- Fee status and payment information, where an authorized data integration is available.

Additional student-specific features will be finalized after further discussion with the college.

Student Permissions

Students may view their own authorized records.

Students must not be able to modify attendance, examination results, fee records, faculty assignments, or other academic information through unauthorized requests.

A student must never be able to access another student's private records simply by changing a roll number or API request.

4.2 Faculty

Faculty members will have individually managed staff accounts.

Login and Security

- Individual username or staff identifier.
- Secure password authentication.
- Password reset and account recovery.
- Optional multi-factor authentication, preferably enabled for privileged accounts.
- Account activation, suspension, and administrative management.

Faculty credentials must not be shared between staff members.

Faculty Dashboard

Faculty members should be able to view:

- Assigned timetable.
- Assigned subjects.
- Assigned branches, classes, and student groups.
- Relevant student roll numbers.
- Attendance management tools.
- Attendance upload history.
- Attendance submission and validation status.

Attendance Responsibilities

Authorized faculty members may:

- Download an attendance spreadsheet template.
- Update attendance records in the spreadsheet.
- Upload the completed spreadsheet.

- Review validation results before submitting.
- Correct records within their assigned permissions.
- View the attendance information for their assigned classes and subjects.

Student Creation

Faculty may be permitted to add new student records where the college authorizes this responsibility.

Student creation must include appropriate validation and must not allow faculty to create duplicate student identities or assign students to unauthorized departments.

The precise student-creation permissions will be configurable according to the college's administrative requirements.

4.3 Head of Department (HOD)

Each HOD will have a secure staff account associated with an authorized department.

HOD Dashboard

The HOD should be able to view:

- Faculty members assigned to their department.
- Student groups and classes within their department.
- Subjects and faculty assignments.
- Department timetables, where required.
- Department attendance summaries.
- Monthly attendance reports.
- Student roll numbers and corresponding attendance percentages.
- Attendance upload and correction status.
- Relevant academic reports.

Attendance Overview

The HOD's main attendance interface should remain simple.

The primary attendance overview should display:

- Student roll number.
- Overall or selected-subject attendance percentage.

The overview should not be cluttered with individual present/absent entries or unnecessary attendance details.

Additional information may be available through a separate detail view when required for verification or authorized correction.

HOD Permissions

The HOD may:

- Monitor attendance within the assigned department.
- Review faculty attendance submissions.
- Request corrections or approve corrections where the college authorizes this workflow.
- Add or manage student records within approved departmental permissions.
- View relevant faculty and academic assignments.

An HOD must not automatically gain access to restricted administrative functions belonging to other departments.

4.4 Principal

The Principal will have the highest institution-wide oversight role.

Principal Dashboard

The Principal should be able to view:

- All authorized departments.
- HOD information and department assignments.
- Faculty members and their department assignments.
- College-wide student statistics.
- Attendance summaries across departments.
- Department-wise attendance comparisons.
- Academic activity and relevant reports.
- Timetable and subject assignment summaries.
- Student-level information when necessary and authorized.

Principal Permissions

The Principal should have institution-wide visibility over the relevant academic and administrative information.

The Principal's dashboard should prioritize summary information and reports rather than displaying every student's detailed records by default.

When necessary, the Principal may navigate to a department, faculty member, class, or student record according to the permissions approved by the institution.

The system should maintain an audit trail for sensitive administrative actions.

5. ROLE-BASED DASHBOARD STRUCTURE

The application will use a single React.js frontend with separate dashboard experiences.

The user will see the dashboard associated with their authenticated role.

5.1 Student Dashboard

Primary sections:

- Overview.
- Student Profile.
- Semester and Subjects.
- Attendance.
- Examinations and Supplies.
- Fees.
- Account Settings.

5.2 Faculty Dashboard

Primary sections:

- Overview.
- My Timetable.
- My Subjects and Classes.
- Students.
- Attendance Management.
- Attendance Uploads.
- Reports.
- Account Settings.

5.3 HOD Dashboard

Primary sections:

- Department Overview.
- Faculty Management.
- Students and Classes.
- Subjects and Assignments.
- Attendance Overview.
- Monthly Reports.
- Academic Administration.
- Account Settings.

5.4 Principal Dashboard

Primary sections:

- College Overview.
- Department Management.
- HOD Management.
- Faculty Overview.
- Student Overview.
- College-Wide Attendance.
- Academic Reports.
- Administrative Settings.

The final navigation structure and visual design will be finalized during the UI/UX design phase.

6. ATTENDANCE MANAGEMENT SYSTEM

Attendance management is one of the central features of the proposed platform.

The goal is to allow faculty members to maintain attendance through familiar spreadsheet workflows while automatically distributing the validated records to the correct students.

6.1 Proposed Attendance Workflow

Step 1: Faculty login.

The faculty member logs in using their individual staff account.

Step 2: Select academic context.

The faculty member selects the relevant department, class, academic period, and subject, subject to their assignments.

Step 3: Download attendance template.

The system provides an Excel-compatible attendance template containing the authorized student roll numbers and required columns.

Step 4: Update the spreadsheet.

The faculty member updates attendance information in the spreadsheet.

Step 5: Upload the completed file.

The faculty member uploads the updated Excel or CSV file through the faculty dashboard.

Step 6: Validate the uploaded records.

The backend checks the file before accepting the data.

Step 7: Preview the results.

The faculty member reviews valid records, warnings, and rejected rows.

Step 8: Confirm submission.

After confirmation, the system saves the validated records and records the submission details.

Step 9: Update student dashboards.

Each affected student's dashboard displays the updated attendance information automatically.

Step 10: HOD monitoring.

The HOD can review the relevant department's attendance percentages and upload status.

Step 11: Principal oversight.

The Principal can view college-wide attendance summaries and relevant department reports.

6.2 Attendance Spreadsheet Structure

The initial spreadsheet may contain the following columns:

- Student Roll Number.
- Student Name, where appropriate.
- Department or Class Identifier.
- Academic Year.
- Semester.
- Subject Code or Subject Identifier, where applicable.
- Attendance Period or Month.
- Total Classes Held.
- Classes Attended.
- Classes Absent.
- Attendance Percentage.

The final columns will depend on whether the college maintains subject-wise attendance, consolidated attendance, or both.

Student names and other descriptive fields may be included for faculty convenience, but the system should use a verified student identifier and academic assignment rather than a name alone to identify the target record.

6.3 Attendance Calculations

The attendance percentage should be calculated using the approved attendance policy.

For a simple total-based calculation:

Attendance Percentage = (Classes Attended / Total Classes Held) × 100

For example:

- Total classes held: 40.
- Classes attended: 36.
- Classes absent: 4.
- Attendance percentage: 90%.

The system should validate that:

- Total classes held cannot be negative.
- Classes attended cannot be negative.
- Classes attended cannot exceed total classes held.
- Classes absent cannot be negative.
- Classes attended plus classes absent must match total classes held when those categories are mutually exclusive and no other attendance status is used.
- Attendance percentage must be calculated consistently.
- Zero-class periods must be handled without division-by-zero errors.

The application should calculate the final attendance percentage on the backend rather than blindly trusting values supplied in an uploaded file.

6.4 Monthly Attendance Management

The faculty workflow should support monthly attendance management.

Faculty may upload a monthly attendance spreadsheet, and the system can generate monthly summaries automatically.

However, the database should preserve the academic period and underlying attendance records or approved totals.

Repeated uploads must not double-count attendance or accidentally erase previous months.

The system must identify whether an upload represents:

- A new attendance period.
- A replacement or correction for an existing period.
- An update to previously submitted records.

The upload process must make these distinctions clear to the faculty member before committing changes.

If the institution requires daily or subject-wise records, the data model must preserve that granularity even when faculty primarily interact through monthly spreadsheets.

6.5 Attendance Validation and Error Handling

The system must identify:

- Unknown roll numbers.
- Duplicate student rows.
- Students outside the faculty member's authorized class.
- Invalid numeric values.
- Missing required fields.
- Incorrect academic periods.
- Invalid or unauthorized subject assignments.
- Duplicate submissions.
- Conflicting updates.
- Inconsistent attendance totals.

The upload interface should provide a clear summary showing:

- Total rows processed.
- Valid rows.
- Rejected rows.
- Rows requiring correction.
- Reasons for rejection.

Invalid rows should not silently disappear.

The system should support a safe retry process without creating duplicate attendance records.

6.6 Attendance History and Audit Trail

The application should maintain an appropriate history of attendance submissions and corrections.

Audit information should include:

- User who performed the action.
- Date and time.
- Affected academic period.
- Affected class or subject.
- Nature of the change.
- Previous and updated values when applicable.
- Submission or correction reference.

The institution should determine who may approve corrections and whether certain attendance periods should be locked after a defined deadline.

6.7 Attendance Reporting

Student dashboard:

- Attendance percentage.
- Monthly attendance summary.
- Subject-wise percentage where applicable.

Faculty dashboard:

- Attendance for assigned classes and subjects.
- Upload history.
- Submission status.
- Relevant student percentages.

HOD dashboard:

- Department-wise student attendance percentages.
- Filters for class, semester, subject, and academic period.
- Monthly department summaries.
- Faculty submission monitoring.

Principal dashboard:

- College-wide attendance summaries.
- Department comparisons.
- Academic-period reports.
- Authorized student-level drill-down when required.

The primary HOD overview should remain limited to student roll numbers and attendance percentages unless further details are requested.

7. STUDENT INFORMATION AND ACCOUNT MANAGEMENT

The system will support student records and user accounts as separate but related concepts.

A student record represents the student's college identity and academic information.

A user account represents the credentials and access permissions used to sign in.

These records must be linked correctly without treating a roll number as a password or storing passwords in plain text.

7.1 Student Record Creation

Authorized faculty members and HODs may be allowed to add student records according to institutional policy.

The system should validate:

- Roll number uniqueness.
- Department assignment.
- Academic year and semester.
- Student enrollment details.
- Required student information.
- Account creation and identity linkage.

The Principal or another designated administrator may be assigned additional account-management responsibilities after approval.

7.2 Student Login Provisioning

The proposed initial login process uses the student's roll number as the identifier and, if approved, as a temporary initial password.

The system should:

- Require the student to change the initial password.
- Store only appropriately hashed passwords.
- Provide secure account recovery.
- Prevent duplicate account creation.
- Link the account to the correct student record.
- Allow administrators to deactivate accounts when necessary.
- Apply rate limits and other authentication protections.

Where existing student accounts or identities are available, the migration process must link the records to the correct portal accounts rather than create unnecessary duplicates.

8. FACULTY, DEPARTMENT, AND SUBJECT MANAGEMENT

The database and application should support the following relationships:

- A department contains authorized student groups and faculty assignments.
- An HOD is associated with the appropriate department.
- Faculty members may teach multiple subjects or classes.
- A subject may be associated with a specific academic year, semester, and department.
- Students are associated with their relevant academic enrollment.
- Faculty timetable entries refer to valid teaching assignments.

These relationships will determine which classes faculty may manage and which records HODs may oversee.

The final academic hierarchy must be confirmed with the institution before implementation.

9. SEMESTER, EXAMINATION, AND SUPPLEMENTARY INFORMATION

The student portal should provide access to semester-related information and supplementary examination details.

The initial scope includes:

- Semester and academic-period information.
- Enrolled subjects.
- Available examination information.
- Supplementary examination information.
- Relevant academic records when available from an authorized source.

The actual result fields, examination workflows, and supply-exam rules will depend on the information maintained by the college and the authorized data sources available.

The project will not assume that a third-party academic system offers an API until that capability has been verified.

10. FEE INFORMATION AND EXISTING PAYMENT INTEGRATION

The institution currently uses an existing fee-payment workflow. The objective is to preserve that workflow wherever technically feasible instead of building a new payment gateway unnecessarily.

The name and technical capabilities of the existing payment platform have not yet been confirmed.

10.1 Expected Existing Payment Flow

The current workflow, as described during initial discussions, is approximately:

1. The student selects the required fee category, such as regular fees.
2. The student enters the required information.
3. A CAPTCHA or other verification step may be required.
4. The student enters a phone number and email address.

5. The student proceeds to the existing payment platform.
6. The student completes the payment through the available payment method.
7. The platform displays a payment result.
8. An invoice or receipt may be sent through email or phone, depending on the existing provider's capabilities.

This is a preliminary description and must be verified against the actual platform.

10.2 Proposed Portal Integration

Once the payment platform is identified, the development team will investigate:

- Whether the platform provides APIs.
- Whether payment status can be retrieved securely.
- Whether it supports signed webhooks or another server-side notification mechanism.
- Whether it provides transaction identifiers and receipt information.
- Whether fee records are already updated automatically.
- Whether the college can authorize an integration.
- Whether the existing payment workflow must remain completely separate.

The portal should not assume a fee has been paid merely because a browser redirects to a success page.

Any integration must rely on an appropriately verified payment status or an authorized reconciliation process.

10.3 Payment Security

The new portal should not collect or store card details or payment credentials unless an explicitly approved, compliant integration requires it.

Payment confirmation must be protected against duplicate processing and forged browser requests.

The payment integration will be finalized after the provider and existing workflow are identified.

11. EXISTING STUDENT-DATA MIGRATION

The project will be developed before the actual student dataset and existing data sources are confirmed.

During initial development, fictional test data will be used.

Once the college authorizes access to the existing records, the team will determine the safest supported migration method.

11.1 Potential Data Sources

The existing records may be available through:

- Excel or CSV exports.
- Existing college management software.
- An existing relational database.
- Authorized reports or APIs.
- A combination of these sources.

The actual source is currently unknown.

11.2 Migration Process

The proposed migration process is:

1. Obtain authorized access to the source records.
2. Identify the fields and relationships available.
3. Map existing fields to the new database schema.
4. Clean and validate the records.
5. Identify duplicates and missing information.
6. Map existing student identities to the correct portal identities.
7. Import data into a test environment.
8. Reconcile record counts and sample records.
9. Verify attendance, academic information, and relevant fee records.
10. Obtain institutional approval for production migration.
11. Import the approved records.
12. Verify the resulting application behavior.

The migration should be repeatable, logged, and designed to avoid duplicate records.

Sensitive records must be transferred through authorized and appropriately secured channels.

11.3 Data Synchronization

If the existing college system continues operating after the new portal launches, the team must determine whether records need ongoing synchronization.

Possible methods include:

- An authorized API integration.
- Scheduled imports.
- An approved data exchange process.

- A controlled manual reconciliation workflow.

The final method will depend on the existing system's capabilities and the college's operating procedures.

12. PROPOSED TECHNICAL ARCHITECTURE

The application will use a separated frontend, backend, and database architecture.

12.1 Frontend

Technology: React.js.

Hosting: Cloudflare Pages.

Responsibilities:

- Responsive user interfaces.
- Role-specific dashboards.
- Login and account interfaces.
- Attendance spreadsheet uploads.
- Attendance summaries and reports.
- API communication.
- User-friendly validation messages.
- Loading, success, and error states.

The frontend must not contain database service-role credentials, backend secrets, or other privileged keys.

12.2 Backend

Technology: Java with Spring Boot.

Hosting: Google Cloud, subject to deployment feasibility and available resource limits.

Responsibilities:

- Authentication and authorization.
- REST API endpoints.
- Role and department permission checks.
- Student and faculty management.
- Academic information services.
- Attendance validation and calculations.
- Spreadsheet import processing.

- Monthly reports.
- Audit logging.
- Payment-provider integration where supported.
- Database access and transaction management.

The backend must enforce permissions on every protected operation. Hiding a frontend button is not sufficient security.

12.3 Database

Technology: Supabase PostgreSQL.

Responsibilities:

- Persistent student and staff records.
- User account references and role assignments.
- Department and academic relationships.
- Subjects and faculty assignments.
- Timetable information.
- Attendance records and summaries.
- Examination information.
- Fee records and verified payment references.
- Upload history and audit records.

Supabase will serve as the database platform. Its additional services may be used where appropriate, but the initial design will keep core business logic and authorization in Spring Boot.

Database credentials must be restricted to the required access level.

12.4 High-Level Request Flow

1. The user opens the React application hosted on Cloudflare Pages.
2. The user authenticates through the approved authentication flow.
3. React sends authorized requests to the Spring Boot REST API.
4. Spring Boot validates the user's identity, role, and resource permissions.
5. Spring Boot reads or updates the relevant PostgreSQL records.
6. The backend returns only the information the user is authorized to receive.
7. React displays the result in the appropriate dashboard.

13. PROPOSED DATABASE ENTITIES

The following entities form an initial conceptual model. The final schema, fields, constraints, and relationships will be defined during technical design.

13.1 Users

Stores login identity references, account status, and role-related information.

Passwords must not be stored in plain text.

13.2 Roles and Permissions

Defines the four primary roles and their authorized actions.

The implementation should allow permissions to be refined without duplicating the application.

13.3 Students

Stores student profiles, roll numbers, academic identifiers, and relevant enrollment references.

13.4 Faculty

Stores staff profiles, department assignments, and account references.

13.5 Departments

Stores branch and department information.

13.6 Subjects

Stores subject identifiers and their academic associations.

13.7 Student Enrollments

Links students to the appropriate academic class, year, semester, and other required academic groupings.

13.8 Faculty Subject Assignments

Links faculty members to the subjects and classes they are authorized to teach.

13.9 Timetables

Stores the relevant teaching schedule and timetable assignments.

13.10 Attendance Records

Stores the attendance data at the level required by the institution, including the appropriate student, academic period, class, and subject references.

13.11 Attendance Uploads

Stores spreadsheet submission metadata, processing results, and relevant upload history.

13.12 Examination and Supply Records

Stores relevant examination information when an authorized source or approved administrative workflow provides it.

13.13 Fee Records

Stores fee obligations and relevant payment status information.

13.14 Payment Transactions

Where supported and approved, stores transaction references and verified payment outcomes without storing sensitive payment credentials.

13.15 Audit Logs

Stores relevant records of sensitive actions, attendance corrections, imports, and administrative changes.

14. SECURITY REQUIREMENTS

Security is a core requirement because the application handles student information, academic records, staff accounts, and potentially fee-related information.

14.1 Authentication

- Secure authentication for all roles.
- Individual staff accounts.
- Forced change of temporary student passwords.
- Password hashing using an appropriate, established mechanism.
- Rate limiting for login attempts.
- Secure account recovery.
- Session expiration and logout behavior.

- Multi-factor authentication for privileged accounts where feasible.

14.2 Authorization

- Students can access only their own records.
- Faculty can access only their assigned classes, subjects, and authorized functions.
- HODs can access their assigned department.
- The Principal can access institution-wide information according to approved permissions.
- Administrative operations require explicit authorization.
- Backend endpoints must enforce the same restrictions regardless of frontend behavior.

14.3 Database Security

- Restrict database credentials and network access.
- Keep privileged secrets on the server.
- Use parameterized database operations or safe ORM queries.
- Enforce appropriate database constraints.
- Protect sensitive records against unauthorized access.
- Configure Supabase Row Level Security where relevant to the chosen database access architecture.
- Avoid exposing privileged database keys to browser code.

14.4 File Upload Security

- Validate file type, size, structure, and content.
- Reject unexpected spreadsheet formats.
- Prevent spreadsheet formula injection in exported files where applicable.
- Enforce record-level authorization.
- Avoid executing uploaded content.
- Log import failures and administrative actions.
- Protect against duplicate submissions and malformed files.

14.5 Audit and Monitoring

The system should maintain logs for relevant security events, privileged actions, attendance corrections, and data imports.

Logs must avoid unnecessarily storing passwords, tokens, or other secrets.

14.6 Cloudflare Security

Cloudflare Pages will host the frontend and provide access to applicable Cloudflare security and edge capabilities.

However, frontend hosting does not automatically secure the backend or database.

Backend authentication, authorization, input validation, secret management, database restrictions, and monitoring remain essential.

14.7 Security Audit Repository

The project owner has an existing security-audit repository intended to test the website and identify vulnerabilities.

The repository will be reviewed during the testing phase.

The proposed process is:

1. Review the audit repository and its documentation.
2. Confirm its supported testing methods.
3. Run appropriate tests against a controlled development or staging environment.
4. Identify and prioritize findings.
5. Fix the confirmed vulnerabilities.
6. Rerun the relevant tests.
7. Perform manual verification of sensitive workflows.
8. Document unresolved risks before release.

Only systems and environments authorized for testing should be targeted.

15. DEPLOYMENT AND INFRASTRUCTURE

15.1 Frontend Deployment

Platform: Cloudflare Pages.

The frontend repository will be connected to the deployment pipeline.

The deployment should support:

- Production builds.
- Environment-specific configuration.
- HTTPS.
- SPA routing configuration.
- Appropriate browser security headers and other supported protections.
- Controlled deployment and rollback procedures.

15.2 Backend Deployment

Platform: Google Cloud.

The team will select the appropriate supported hosting service for Spring Boot after evaluating operational requirements and free-tier eligibility.

The deployment should support:

- A secure API endpoint.
- Environment-based configuration.
- Database connectivity.
- Application logs.
- Health checks.
- Appropriate resource limits.
- Secure secret configuration.
- Restart and recovery procedures.

The architecture should not assume that all Google Cloud resources are free. Current quotas, free-tier eligibility, billing requirements, and expected usage must be checked before production deployment.

15.3 Database Deployment

Platform: Supabase PostgreSQL.

The database configuration should include:

- Appropriate schema and constraints.
- Restricted credentials.
- Secure connectivity.
- Backup and recovery planning.
- Migration scripts.
- Development and production environment separation where feasible.
- Monitoring of storage and usage limits.

15.4 Environment Management

Development, staging, and production configurations should be separated where practical.

Production student records must not be copied into local development environments without an approved need and appropriate protection.

16. FUNCTIONAL REQUIREMENTS

The system should satisfy the following requirements.

FR-01: Users can authenticate through the appropriate role-specific login process.

FR-02: Each authenticated user sees the dashboard associated with their authorized role.

FR-03: Students can view their own academic and attendance information.

FR-04: Faculty can view their assigned classes, subjects, and timetable.

FR-05: Authorized faculty and HODs can create or manage student records within approved permissions.

FR-06: Faculty can download the attendance template.

FR-07: Faculty can upload an Excel or CSV attendance file.

FR-08: The system validates uploaded student identifiers and attendance values.

FR-09: The system displays a preview before accepting an attendance submission.

FR-10: Confirmed attendance records are associated with the correct student identities.

FR-11: Student dashboards display the latest accepted attendance information.

FR-12: The system calculates attendance percentages using the approved rules.

FR-13: HODs can view department-level student attendance percentages.

FR-14: The Principal can view college-wide attendance and organizational summaries.

FR-15: The system maintains appropriate attendance submission and correction history.

FR-16: Duplicate uploads must not create duplicate attendance records.

FR-17: The application supports authorized import of existing student records.

FR-18: Fee integration is implemented only after the existing payment platform is identified and its integration capabilities are verified.

FR-19: The application is deployed using the approved frontend, backend, and database infrastructure.

FR-20: Security and functional testing are completed before the production launch.

17. NON-FUNCTIONAL REQUIREMENTS

17.1 Performance

The application should load quickly and provide responsive interactions under the expected college workload.

Database queries should be indexed appropriately, and pagination should be used for large student lists.

Bulk uploads should be processed without unnecessary repeated database requests.

Performance targets will be established after the expected number of users, students, and concurrent sessions is confirmed.

17.2 Reliability

The application should handle invalid requests and failed imports without corrupting existing records.

Database updates should use appropriate transactions.

Failed operations should return clear errors and support safe recovery.

17.3 Usability

The interface should be simple and consistent.

Faculty attendance uploads should require as few unnecessary steps as possible.

HOD and Principal dashboards should emphasize summary information, filters, and useful reports.

17.4 Accessibility and Responsiveness

The application should support common desktop and mobile screen sizes and follow appropriate accessibility practices.

17.5 Maintainability

The codebase should use a clear project structure, documented APIs, database migrations, automated tests, and environment-specific configuration.

17.6 Privacy

Student and staff information must be processed only for authorized institutional purposes.

Access to private records should be limited to the information required for each role.

18. USER INTERFACE AND EXPERIENCE REQUIREMENTS

The application should have a professional, modern, and consistent visual identity appropriate for an engineering college.

The final visual design will be created after the functional structure is agreed upon.

Expected UI characteristics:

- Responsive layouts.
- Clear navigation.
- Consistent typography and spacing.
- Accessible color contrast.
- Readable data tables.
- Search and filtering for student lists.
- Clear upload progress and validation feedback.
- Confirmation prompts for important changes.
- Empty, loading, error, and success states.
- Simple attendance summary cards.
- Role-specific dashboard navigation.

The design should prioritize clarity and efficiency over unnecessary visual complexity.

19. IMPLEMENTATION PLAN

Development will proceed in phases so that the core architecture is established before complex integrations are attempted.

Phase 1: Requirements and Approval

- Review this PRD with the HOD and relevant college representatives.
- Confirm role permissions.
- Confirm the attendance policy.
- Confirm whether attendance is subject-wise, consolidated, or both.
- Identify required student fields.
- Confirm who can create students and modify attendance.
- Document unresolved requirements.

Deliverable: Approved requirements and scope.

Phase 2: Architecture and Database Design

- Define the application architecture.

- Design the PostgreSQL schema.
- Define relationships between students, departments, faculty, subjects, and attendance.
- Define the authentication approach.
- Define role and resource permissions.
- Design the API structure.
- Define migration and audit requirements.

Deliverable: Technical design and initial database migrations.

Phase 3: Project Foundation and Security

- Initialize the React frontend.
- Initialize the Spring Boot backend.
- Configure Supabase PostgreSQL connectivity.
- Configure environment variables and secrets.
- Implement authentication and authorization.
- Add baseline security controls.
- Establish automated testing.

Deliverable: Secure application foundation.

Phase 4: Core Dashboards

- Build the student dashboard.
- Build the faculty dashboard.
- Build the HOD dashboard.
- Build the Principal dashboard.
- Implement role-specific navigation.
- Connect the dashboards to backend APIs.

Deliverable: Functional role-based portal using fictional test data.

Phase 5: Academic and Attendance Management

- Implement student and faculty management.
- Implement subjects, class assignments, and timetable relationships.
- Build attendance templates.
- Implement spreadsheet uploads.
- Implement validation and preview.
- Implement attendance calculations.
- Implement monthly reports.
- Implement audit logging and correction workflows.

Deliverable: Tested attendance management system.

Phase 6: Academic Information and Fee Investigation

- Implement available semester and supplementary examination information.
- Identify the source of existing student records.
- Identify the existing fee-payment provider.
- Evaluate available APIs and authorized exports.
- Determine the fee reconciliation process.
- Define integration requirements.

Deliverable: Academic information workflows and approved integration plans.

Phase 7: Data Migration

- Obtain authorized source data.
- Map and clean records.
- Test imports in a non-production environment.
- Reconcile sample records.
- Verify student identity mapping.
- Validate attendance and relevant academic relationships.
- Perform the approved production migration.

Deliverable: Verified production records.

Phase 8: Deployment

- Configure Cloudflare Pages.
- Deploy Spring Boot to the selected Google Cloud service.
- Configure Supabase production access.
- Configure secrets, HTTPS, logging, and monitoring.
- Test production connectivity.
- Document deployment and rollback procedures.

Deliverable: Deployed application.

Phase 9: Security Audit and Acceptance

- Run the security-audit repository against an authorized test environment.
- Conduct functional and integration tests.
- Test role-based access restrictions.
- Test spreadsheet imports and duplicate handling.
- Test authentication and account recovery.
- Resolve critical and high-priority findings.
- Obtain institutional acceptance.

Deliverable: Release candidate ready for authorized production use.

20. ACCEPTANCE CRITERIA

The project will be considered ready for institutional acceptance when the approved scope satisfies the following criteria.

Authentication

- Each role can authenticate through its approved process.
- Users cannot access unauthorized dashboards or APIs.
- Students cannot retrieve another student's private information.
- Staff permissions are enforced on the backend.

Attendance

- Faculty can download and upload the approved spreadsheet format.
- Invalid records are identified clearly.
- Valid records are linked to the correct students.
- Repeated uploads do not silently duplicate or overwrite unrelated records.
- Attendance calculations follow the approved policy.
- Students can see their accepted attendance records.
- HODs can view the required department-level percentages.
- The Principal can view college-wide summaries.

Administration

- Authorized staff can manage the records permitted by their role.
- HOD access is restricted to the appropriate department.
- Principal access is configured according to institutional approval.
- Important administrative changes are logged.

Migration

- Imported records are validated against the approved source.
- Duplicate student identities are resolved.
- The migration results are reconciled before production use.
- Failed imports are reported and can be safely retried.

Deployment and Security

- The frontend, backend, and database communicate securely.
- Production secrets are not exposed in frontend code.
- Automated and manual tests cover critical workflows.
- The agreed security checks are completed and findings addressed according to severity.

- Backup and recovery responsibilities are documented.

21. PROJECT SCOPE

21.1 Initial Scope

The proposed initial release includes:

- Four role-based dashboards.
- Secure authentication and authorization.
- Student profiles and academic information.
- Faculty, HOD, and Principal management views.
- Subject and timetable relationships.
- Spreadsheet-based attendance management.
- Attendance calculations and monthly summaries.
- Authorized student creation.
- Initial deployment and security testing.

21.2 Conditional Scope

The following depend on institutional requirements or existing-system capabilities:

- Importing real student records.
- Synchronizing records from the old college system.
- Automated fee-payment status updates.
- Payment transaction integration.
- Invoice and receipt integration.
- Automated examination-result synchronization.
- Additional student services not yet specified.

21.3 Out of Scope for the Initial Release

Unless separately approved, the initial release will not include:

- Building a new payment gateway.
- Storing card credentials.
- Replacing the college's existing payment provider.
- Bypassing existing college-system access controls.
- Importing unverified or unauthorized personal data.
- Implementing unspecified features before requirements are confirmed.

22. RISKS AND MITIGATION

22.1 Unknown Existing Data Source

Risk: Existing student records may be stored in an unfamiliar or inaccessible system.

Mitigation: Develop against fictional data first, then assess authorized exports, APIs, or database access.

22.2 Incorrect Attendance Uploads

Risk: A spreadsheet may contain incorrect roll numbers, duplicate rows, or invalid totals.

Mitigation: Validate the file, show a preview, reject invalid records, and preserve upload history.

22.3 Unauthorized Access

Risk: A user may attempt to access records outside their role.

Mitigation: Enforce backend authorization, restrict database access, and perform security testing.

22.4 Payment Integration Uncertainty

Risk: The existing payment platform may not support automated updates or provide a suitable API.

Mitigation: Identify the provider, verify its supported integration methods, and retain an approved reconciliation workflow if automation is unavailable.

22.5 Hosting Limits and Costs

Risk: Hosting quotas or usage limits may affect reliability or cost.

Mitigation: Verify current Google Cloud and Supabase limits, estimate usage, and document resource and billing requirements before production.

22.6 Incomplete Institutional Requirements

Risk: Academic policies and role responsibilities may differ from initial assumptions.

Mitigation: Obtain HOD or institutional approval for the role matrix, attendance policy, data fields, and operational workflows.

23. OPEN QUESTIONS REQUIRING INSTITUTIONAL CONFIRMATION

The following decisions should be confirmed before finalizing the implementation.

1. What exact student fields must be maintained?
2. Is attendance subject-wise, consolidated, or both?
3. Does the college officially calculate attendance daily, monthly, or through another academic period?
4. Which staff members may add students?
5. Who can approve attendance corrections?
6. Should HODs only view attendance percentages or also approve corrections?
7. What additional academic features must the student dashboard contain?
8. Where are the current student records stored?
9. What is the existing fee-payment platform?
10. Does the payment platform provide an API or automated payment notifications?
11. Does the existing system update fee records automatically?
12. What authentication and account-recovery process does the college require?
13. What are the expected number of students, faculty members, and concurrent users?
14. What hosting, backup, privacy, and operational requirements must be met?
15. Who will approve the application for production use?

24. FINAL RECOMMENDATION

The recommended approach is to build the application foundation and role-based dashboards first, then implement attendance management and the required academic workflows.

The initial system should use fictional test records and a well-defined database schema. This allows the team to develop and test the application without waiting for access to the college's existing student dataset.

The attendance workflow should prioritize spreadsheet uploads, automatic validation, reliable record mapping, and simple dashboard summaries. Faculty should not need to manually enter every student record into the website when an approved spreadsheet workflow can achieve the same objective.

HODs should receive a clear department-level overview, primarily showing student roll numbers and attendance percentages. The Principal should receive institution-wide oversight, with additional details available through authorized drill-down views.

The existing fee-payment workflow should be investigated and integrated only after its technical capabilities are verified.

Security must be incorporated throughout development rather than treated as a final step. The existing security-audit repository should be included in the testing process, alongside authentication, authorization, data-validation, and integration tests.

The final implementation should be based on confirmed institutional requirements and approved access to college records.

The intended outcome is a unified, maintainable, and secure college management platform that simplifies academic workflows while preserving the integrity and privacy of student and staff information.

Document Status: Initial proposal for HOD review and institutional discussion.

Next Step: Review the requirements with the HOD, confirm the academic and administrative workflows, and approve the initial scope before implementation begins.